CPSC 321 project — step 1

proposal : steam library value & backlog tracker

Greyson Knippel

November 5, 2026

Proposal

theme

My project is a database for a web application that helps PC gamers (specifically ones on the "steam" platform) get more out of the games they already own and make smarter decisions about what to buy next. Many steam users buy games on sale and never play them, or play some games for hundreds of hours and others for twenty minutes. The application answers three questions: *which of my games were actually worth the money? what should I play next from my backlog?* and *what should I buy next, based on the games that gave me the best value?*

data stored and why

my database combines two kinds of data:

- **catalog data from steam** (via the steam web API, steam store API, and steam spy): games, developers, publishers, genres, user defined tags, platforms, release dates, review scores, and current prices. Prices are stored over time as a price history, since steam prices change frequently with sales.
- **user data owned by the application**: user accounts, the games each user owns, the price each user actually paid and when, total hours played (imported from the steam API's owned games endpoint), and a personal backlog of games the user intends to finish, with a priority, the date added, a status (*backlog*, *playing*, *completed*, *dropped*), and an optional target completion date.

Catalog data is mostly read only, while user data is what the application adds, updates, and removes. Storing the price *paid* (not just the store price) allows the application to compute each game's **cost per hour** (price paid ÷ hours played), which is the entire "worth it" thing measure. And storing tags allows the application to find games that are similar to one another.

application

The database is going to be built on a web application (python, psycopg, and streamlit) with three main features:

1. **value report:** shows the cost per hour of each game a user owns, summarizes value by genre and tag, and flags games that were poor value compared to the rest of the user's library.
2. **backlog manager:** lets users add games to a backlog, update their progress, remove games, and filter and sort the backlog (e.g., by how many days a game has been waiting, or how many hours they have on it).
3. **recommendations:** suggests games the user does not own that share many tags with the user's best valued games, filtered by price and rating.

the planned design will include at least the following tables: user, game, developer, publisher, genre, tag, purchase, backlogEntry, and priceHistory, with relationships between users and games (through purchases and backlog entries), games and tags, and games and genres.

Use Cases

UC1: import or record a game purchase

A user records a game they own: the game, the price they paid, and the purchase date. Hours played are imported from the user's steam account (or entered manually if desired). If the user entered a purchase by mistake, they can delete it.

justification: cost per hour cannot be computed without knowing what the user actually paid, which steam does not provide. every value report and recommendation depends on this data.

operations: adding data / removing data.

UC2: add a game to the backlog

A user adds a game they own to their backlog with a priority (1–5) and an optional target completion date. The date added is recorded automatically.

justification: the backlog manager's purpose is to track games the user intends to play. recording the date added makes it possible to sort and filter by how long each game has been waiting.

operations: adding data.

UC3: update backlog progress and playtime

A user changes a backlog game's status (e.g., from *backlog* to *playing*, or *playing* to *completed*), changes its priority, or refreshes their hours played from steam. When a game is marked completed, its completion date is recorded.

justification: the backlog is only useful if it reflects the user's current progress, and cost per hour changes as the user plays more.

operations: updating data.

UC4: remove a game from the backlog

A user removes a game from their backlog because they no longer intend to play it. The purchase record remains, so the game still counts toward the value report.

justification: users' plans change, keeping abandoned games in the backlog would make sorting and filtering less useful.

operations: removing data.

UC5: search the backlog

A user filters their backlog on several conditions at once and sorts the results, for example: *games tagged "Roguelike" that have been in the backlog for more than 90 days, have a priority of 3 or higher, and cost less than $15, sorted by days in backlog (longest first).*

justification: a large backlog is hard to manage; filtering by tag, time waiting, priority, and price helps a user choose what to play next.

operations: searching data (faceted: tag, days in backlog, priority, price paid, status).

UC6: search the game catalog

A user searches the store catalog with several filters, for example: *games under $20 tagged "metroidvania" with at least 85% positive reviews, released after 2020, that run on Linux and that the user does not already own.*

justification: before buying, users want to narrow the catalog to games that match their budget, preferences, and hardware.

operations: searching data (faceted: price, tag, genre, review score, release date, platform, ownership).

UC7: value report by genre and tag

For a given user, the application computes the average cost per hour, total amount spent, and total hours for each genre, and lists only the tags where the user has spent more than $50 but averaged fewer than 10 hours per game (categories where the user tends to overpay). It also lists the user's "regret" games: those whose cost per hour is higher than the user's average cost per hour across their whole library.

justification: this is the core "was it worth it?" feature and helps users notice spending patterns before their next purchase.

operations: analyzing data. covers question kinds (1) summary per group (per genre), (2) groups meeting a condition (tags with `HAVING` on spending and hours), and (3) comparison against a computed value (games above the user's average cost per hour).

UC8: recommend similar high valued games

The application finds the user's "worth it" games (cost per hour below the user's average), counts the tags those games have in common, and recommends games the user does not own that share at least three of those tags, ranked by the number of matching tags and filtered by a maximum price.

justification: users get the most value from games similar to ones that already gave them many hours for their money, so these are the strongest recommendations the application can make.

operations: analyzing data. covers question kinds (1) summary per group (tag counts across the user's best value games), (2) groups meeting a condition (games sharing at least three tags, using `HAVING`), and (3) comparison against a computed value (games below the user's average cost per hour, using a subquery).

UC9: track price history

The data loading script periodically records each game's current steam price, and the application shows each game's lowest recorded price and whether the current price is below that game's average historical price.

justification: users buying games for good value want to know whether a sale is actually a good deal.

operations: adding data (new price records); analyzing data (question kinds (1) and (3)).

use case	add	remove	update	search	analyze
UC1 record purchase	✓	✓			
UC2 add to backlog	✓				
UC3 update progress			✓		
UC4 remove from backlog		✓			
UC5 search backlog				✓	
UC6 search catalog				✓	
UC7 value report					1, 2, 3
UC8 recommendations					1, 2, 3
UC9 price history	✓				1, 3